



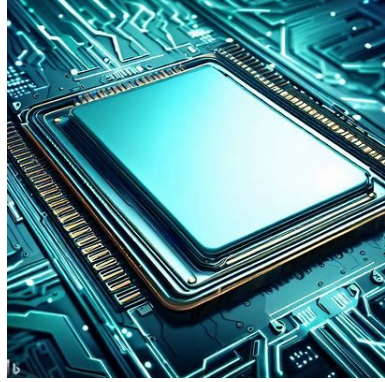
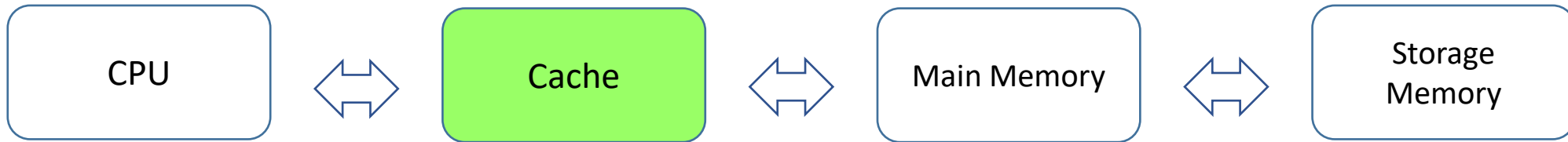
SC1006

Computer Organisation and Architecture

Cache

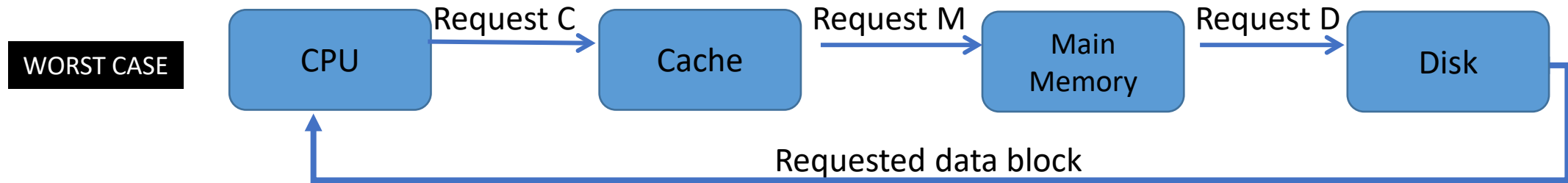
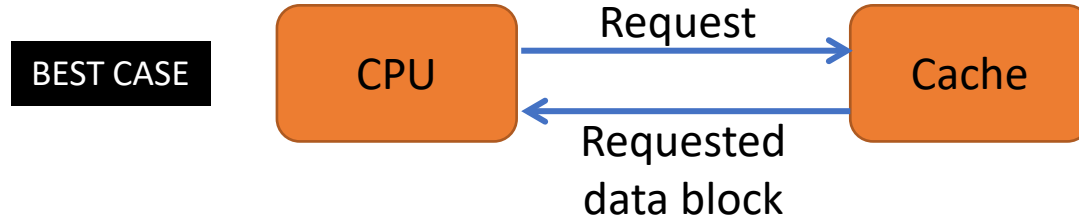
Prof Lin WeiSi
College of Computing and Data Science, NTU
Email: wslin@ntu.edu.sg

Cache



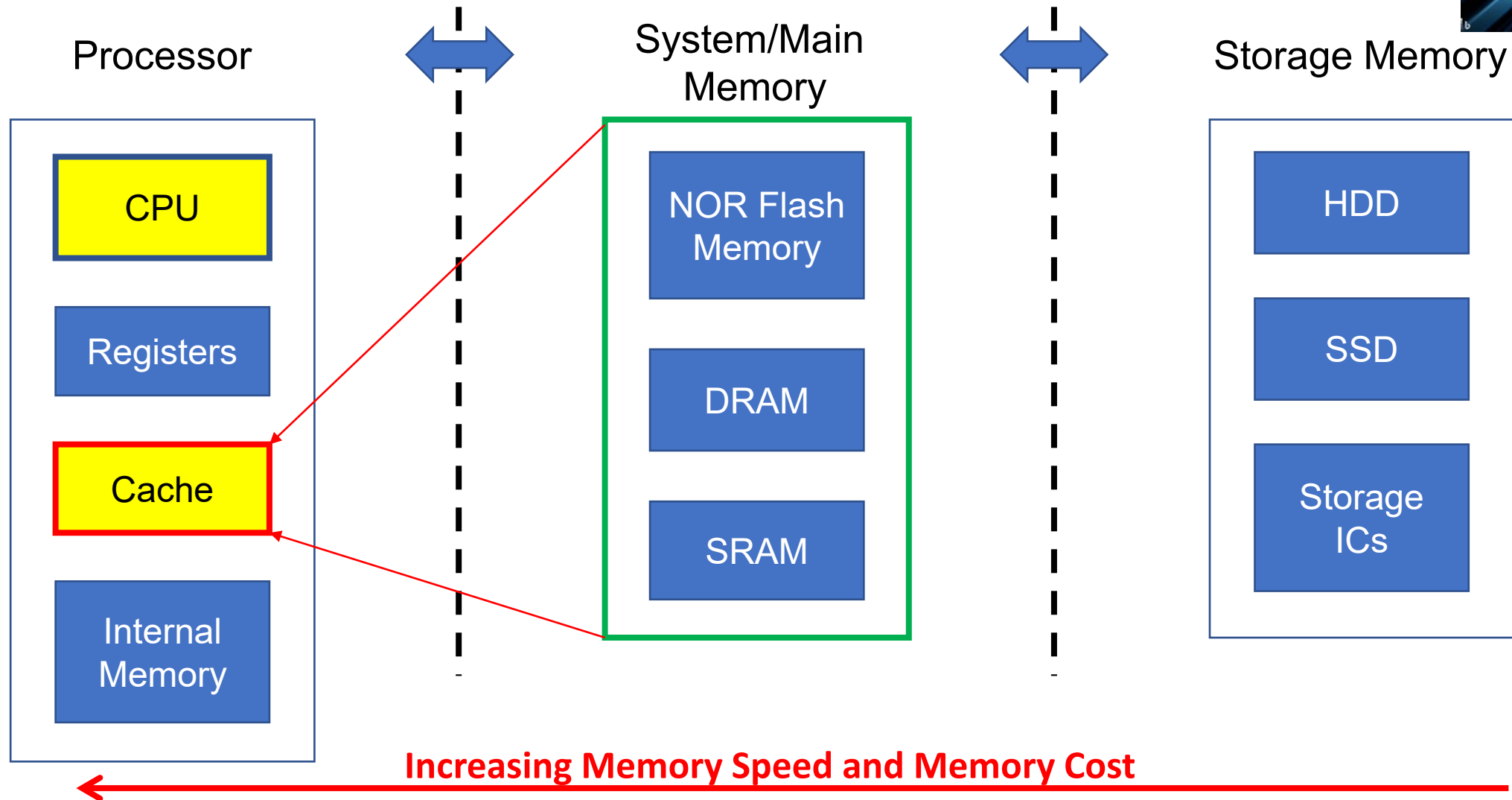
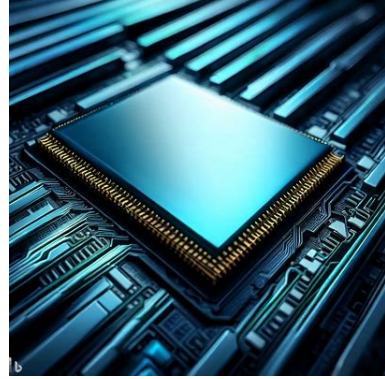
- Issue: **CPU** speed is typically much **faster** than **external memory**.
 - CPU speed in Ghz region
 - External Memory Speed in 100s of Mhz region.
- **Need a fast memory** to act as a **buffer** between the Main memory and CPU.
- The purpose of cache memory is to speed up accesses by storing/fetching **recently used data** closer to the CPU instead of the main memory (slower access).
- Potentially able to **improve the overall system performance** drastically.

CPU Memory Access

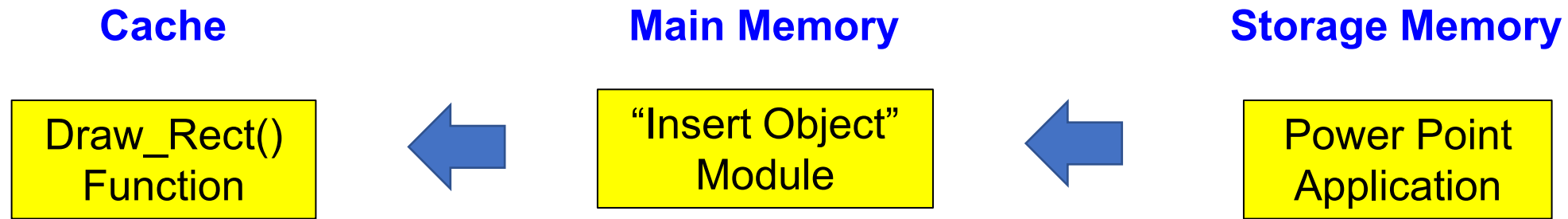


- To access a particular piece of data, the CPU first sends a request to its nearest memory, usually cache.
- If the data is not in cache, a query is then sent to the main memory.
- If the data is not in main memory, then the request goes to disk.
- Once the data is located, the **required data and a number of its nearby data elements** are fetched into cache memory simultaneously.

Computer Memory (Programmer's View)



Cache Design - Introduction



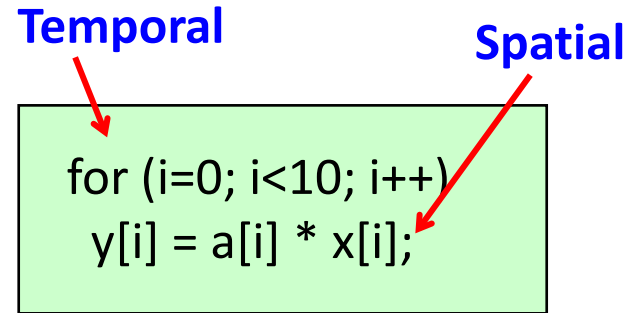
- Example: Power Point Application
- Entire Application is stored in Storage Memory
- The user is trying to insert some object into the powerpoint slide
 - The corresponding code for object insert is transferred to Main memory for execution
- At the instance, the user is drawing a rectangle
 - The corresponding code for the Draw_Rect() function finds its way into the cache eventually.

Principle of Locality

- Now that we know cache only needs to contain a subset of the main memory to work
- In what way should the data be transferred between main memory and cache in order to maximise the efficiency of a cache?
- Efficiency meaning how much improvement can the cache bring to the system. This is related to the probability the CPU is able to find the information that it needs in the cache.
- For that, we need to understand the attributes of how information is organised and accessed when a program is executed.
- This is summarised in the concept called Principle of Locality



Principle of Locality



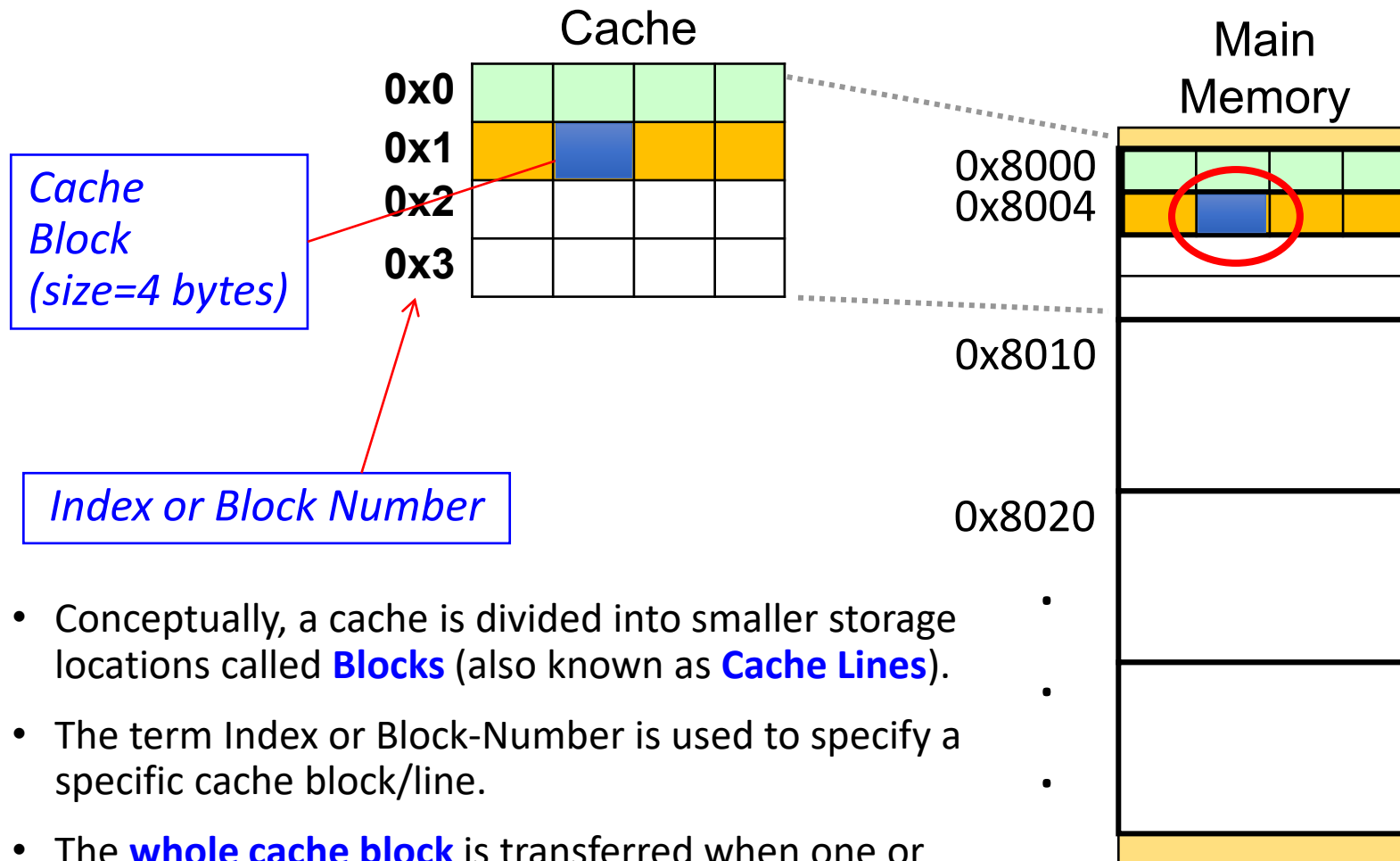
- The **principle of locality** tells us that once a byte in a program is accessed, it is likely a **nearby data element** will be needed soon.
- There are two principles of locality governing this behaviour
- Locality of **Space** (or Spatial Locality)
 - **Code/Data that is nearby each other** is likely to be accessed together
 - Transfer of data between main memory and cache is done in blocks to leverage on this behaviour
- Locality of **Time** (or Temporal Locality)
 - **Recently accessed code/data** is more likely to be accessed again
 - Used to decide which item to replace in the Cache

Cache Mapping Scheme

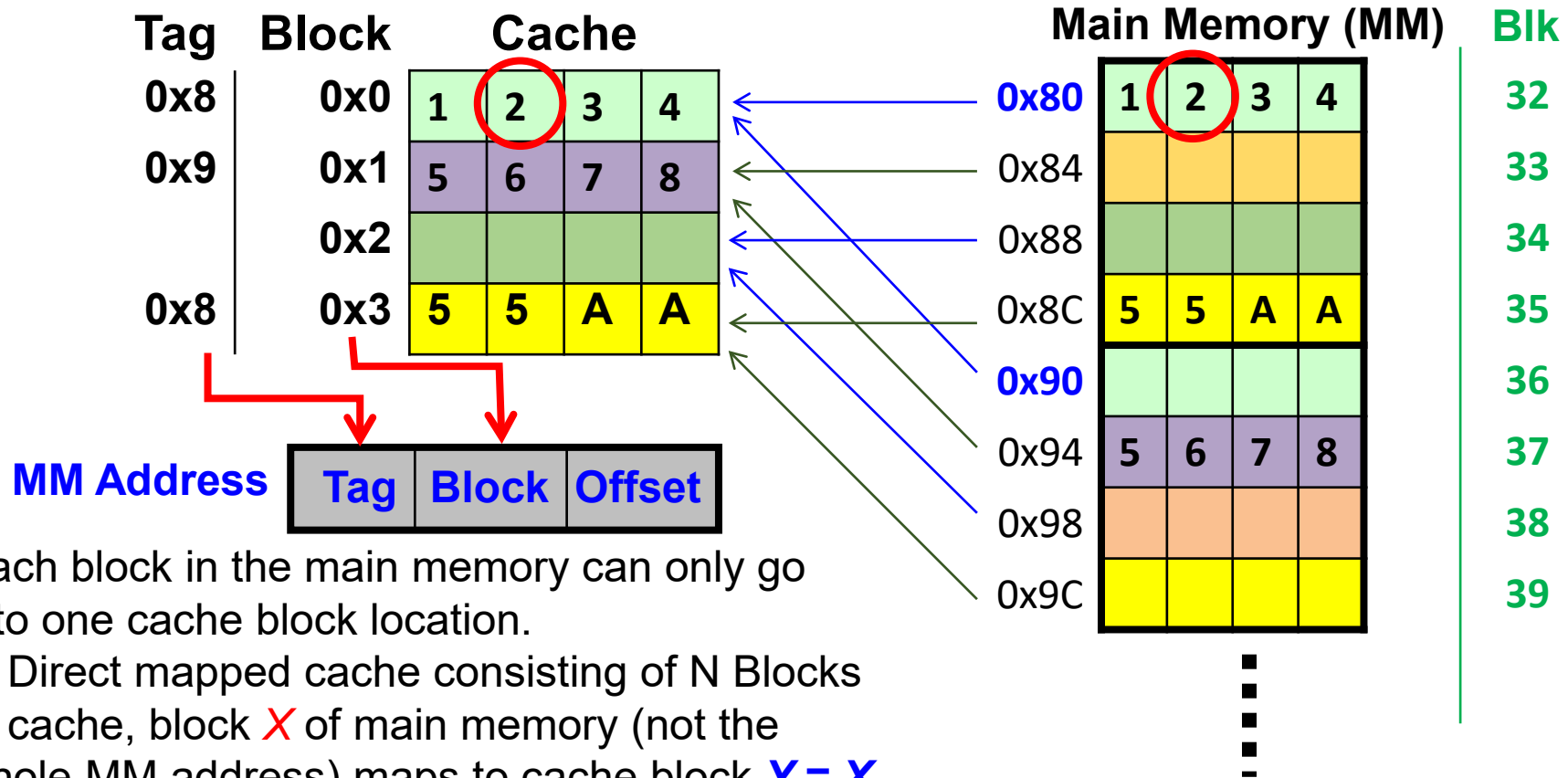
- We understand now that data transfer between main memory and cache is done in blocks instead of individual bytes to leverage on the principle of locality.
- Another attribute of cache design that affects the efficiency of the cache is the cache mapping scheme.
- Cache mapping scheme deals with how each main memory block is mapped to a particular cache block, e.g. Main memory block #0x80 is mapped to cache block #0 (index 0).
- There are three basic cache mapping schemes
 - Direct Mapped
 - Set Associative
 - Fully Associative
- We will only touch on Direct Mapped Cache in this course, rest of the mapping scheme will be discussed in Advanced Computer Architecture Course.



Terms used in Cache Mapping



Direct Mapped Cache



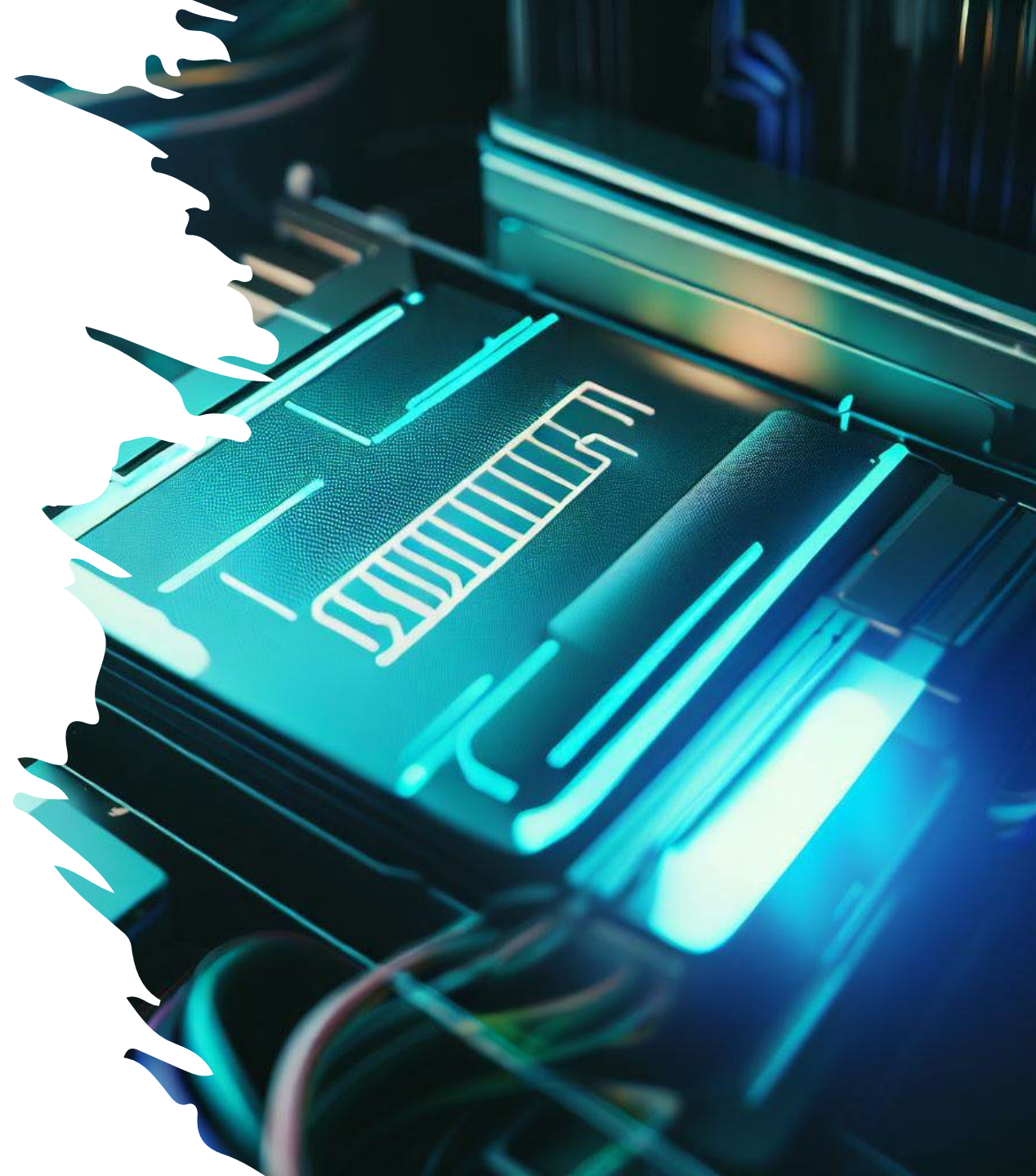
- Each block in the main memory can only go into one cache block location.
- In Direct mapped cache consisting of N Blocks of cache, block **X** of main memory (not the whole MM address) maps to cache block **$Y = X \bmod N$** .
- Whole block of data is filled when one or more bytes in the block is needed.
- A **Tag value** keeps track of which main memory block is associated with each cache block.

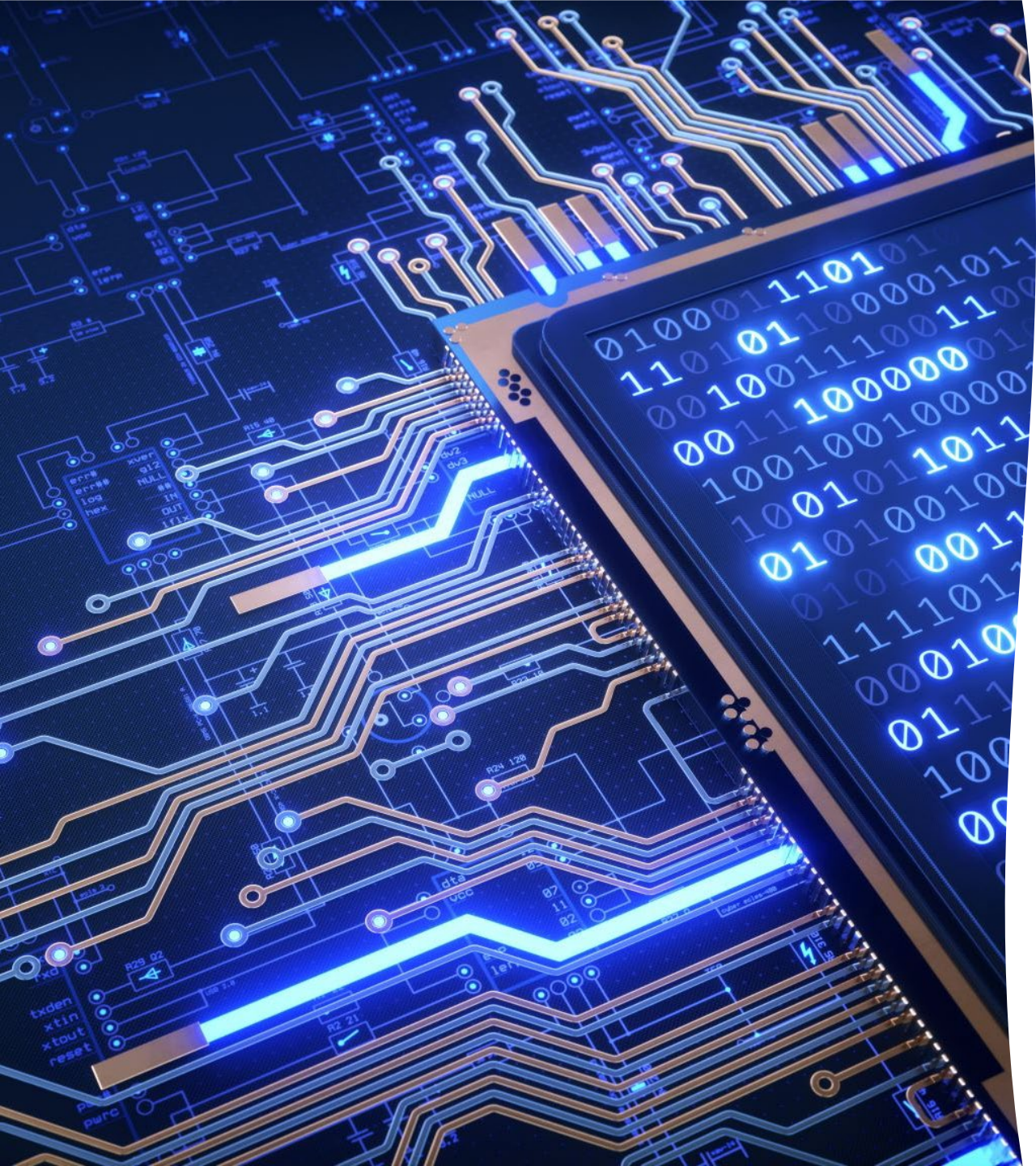
0x81 = 1000 00 01

Tag=0x8 Block=0x0 (X) Offset=0x1

Cache Mapping (Elaboration on basic principle)

- Cache mapping
 - Allocates the data in the entire main memory into the cache which is of a much smaller size.
 - Able to uniquely identify each and every main memory location within the cache via the cache way of addressing: Block Index, Offset of the data within the block, and the corresponding Tag Value of the block.
- Since the CPU will use the MM address as the reference to retrieve the information, it is only natural that the MM address is used as the basis to enable the mapping process.



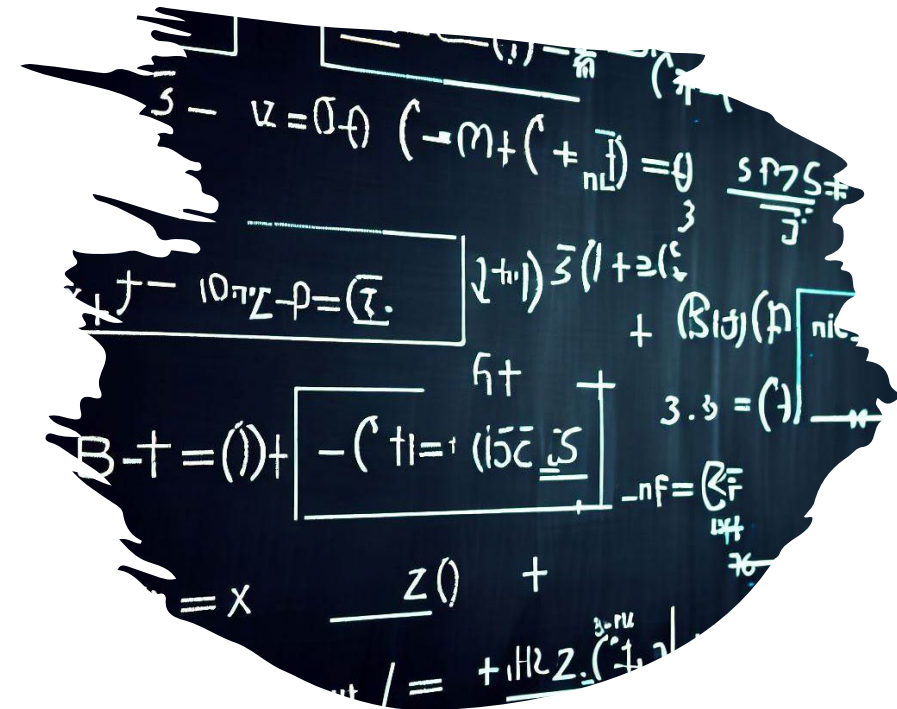


Cache Mapping (Elaboration on basic principle)

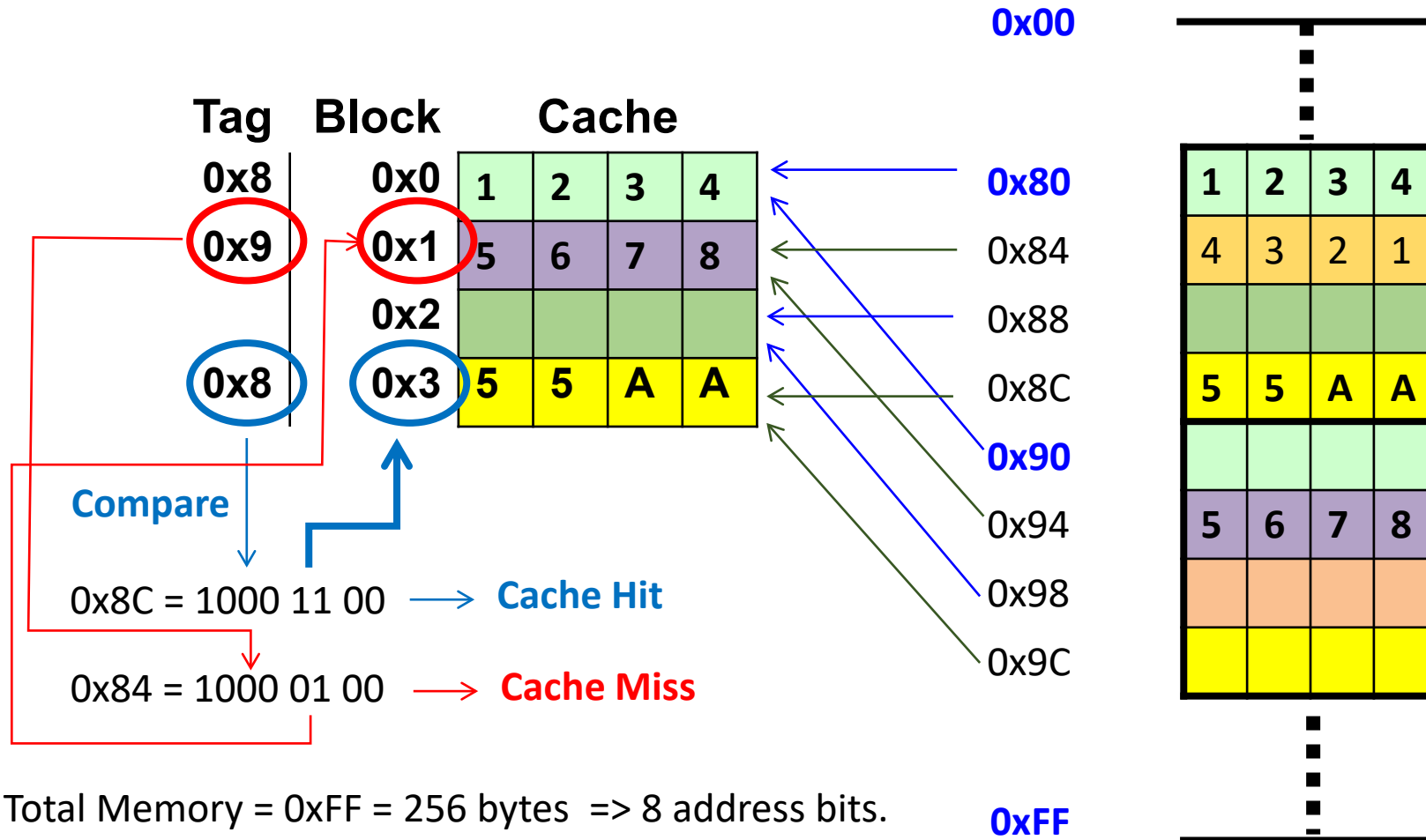
- The **number of bits allocated to each field** is a result of the structure of the cache.
 - **Offset** refers to the targeted data's location offset from the start of the cache block. Something like an address within the cache block. So if the size of a cache block is 16, one would need 4 bits in the OFFSET field to address these 16 locations.
 - **Block** refers to the index of the cache block that the targeted data will be mapped to. If there are 8 cache blocks for example, then one would need 3 bits in the BLOCK field to fully represent the cache block index.
 - **Tag** bits are the left over of target data's main memory address after partitioning for Offset and Block Index. Having this information will allow the cache system to **uniquely identify** the data in the cache.

Direct-Mapped Cache Mapping Example

- Given a system with following attribute
 - Main/System Memory size = 64KBytes**
 - Cache Size = 256Bytes**
 - Cache Block Size = 16Bytes**
- Where would Data at main memory address **0x1106** be mapped to?
- Derivation of cache mapping format
 - Cache Block Size = 16Bytes $\Rightarrow$ **#Offset bits = $\log_2(16) = 4$ bits.**
 - Number of Cache Blocks = $256/16 = 16$ Blocks
 $\Rightarrow$ **#Blk bits = $\log_2(16) = 4$ bits.**
 - Main memory size = 64KBytes $\Rightarrow \log_2(64*1024) = 16$ bits address.
#Tag bits = $16-4-4 = 8$ bits
 - Mapping Format = 8:4:4
- Applying to the main memory address 0x1106
 - 0x1106 = **0001 0001 0000 0110b**
 - BLK = 0x0, OFFSET=0x6, TAG=0x11**



Data Retrieval Example (Direct-Mapped Cache)



Total Memory = 0xFF = 256 bytes => 8 address bits.

Cache Size = 16Bytes

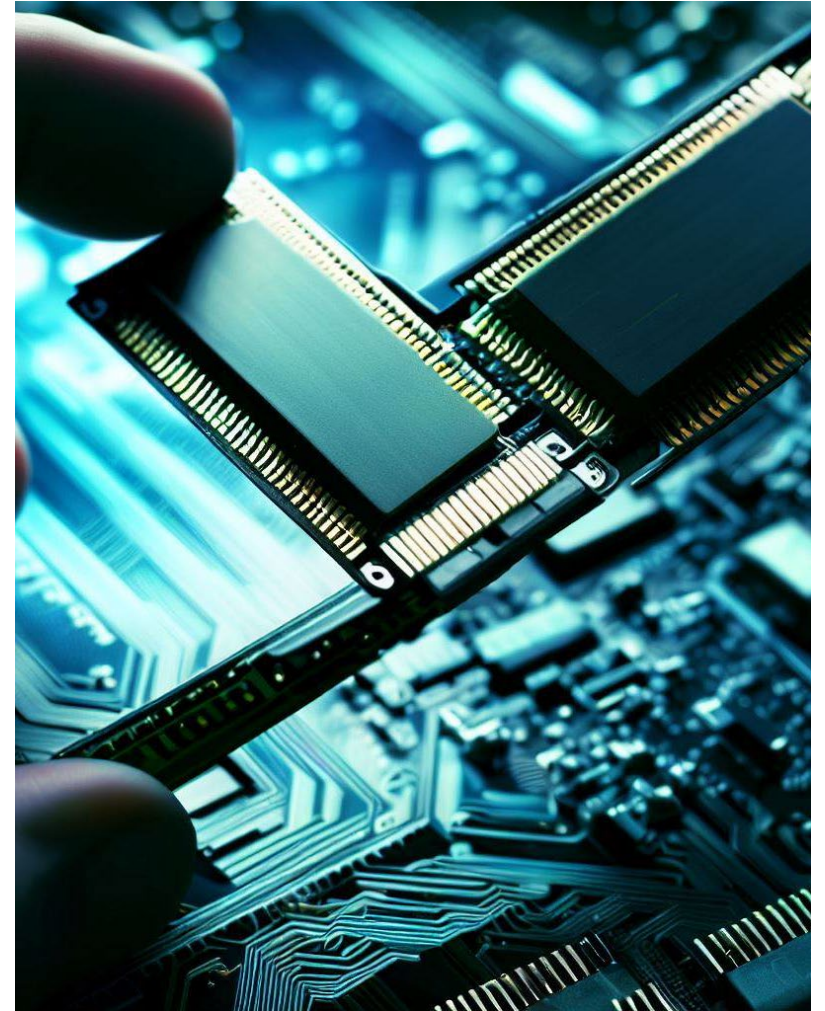
Cache Block Size = 4 Bytes => 2 address bits for offset.

of Blocks in Cache = 16/4 = 4 Blocks => 2 address bits (Block index)

of TAG bits = 8-2-2 = 4 bits.

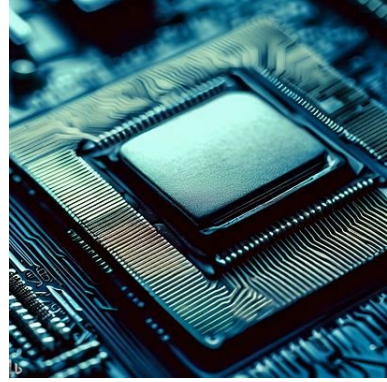
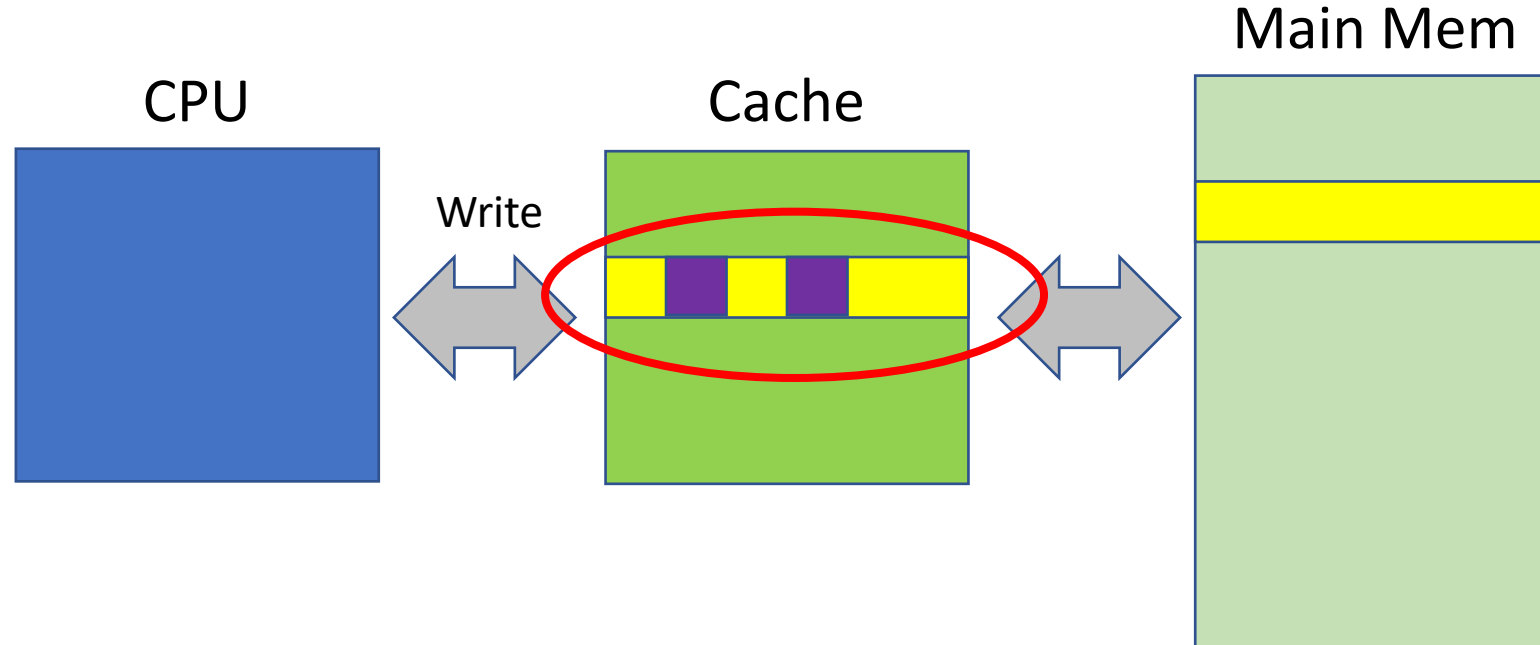
Cache Memory Replacement Policy

- The direct mapped cache doesn't really need a cache replacement policy as its mapping is one-to-one.
- When there is a need to transfer a block of data to the cache but the cache is fully occupied, there is a need to decide which cache block to evict/purge in order to free up space for the new data.
- The algorithm used to decide which block gets evicted is designed based on some Cache Replacement Policy. Two examples illustrated below
- **Least recently used (LRU)** algorithm keeps track of the last time that a block was assessed and evicts the block that has been unused for the longest period of time. The disadvantage of this approach is its complexity: LRU has to maintain an access history for each block, which ultimately slows down the cache.
- **First-in, first-out (FIFO)** is a popular cache replacement policy. In FIFO, the block that has been in the cache the longest, regardless of when it was last used.

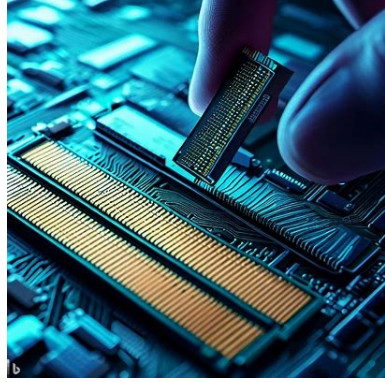


Cache Write Policy

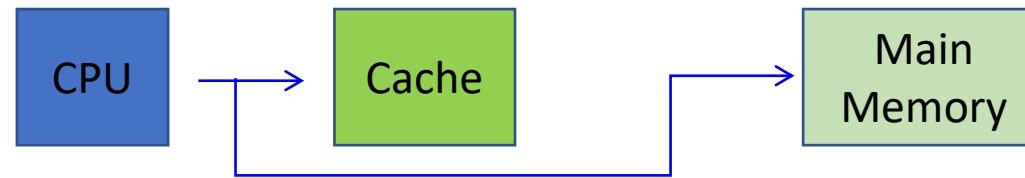
- Locations in cache that are written into by CPU is known as **Dirty Block**
- Cache replacement must take into account **dirty blocks**, those blocks that have been updated while they were in the cache.
- Dirty blocks must be written back to memory. A **write policy** determines how this will be done.
- There are two types of write policies: **write through** and **write back**.



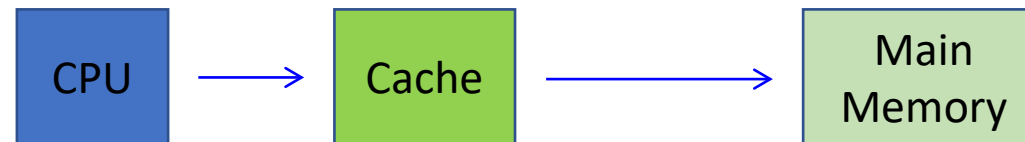
Cache Write Policy (Write Hit)



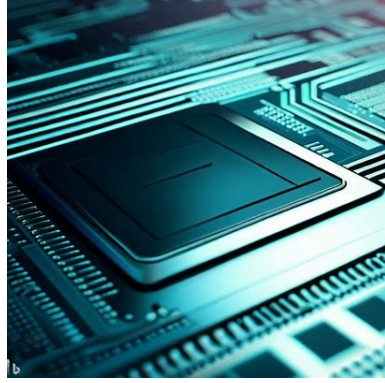
- **Write through** updates cache and main memory **simultaneously** on every write.



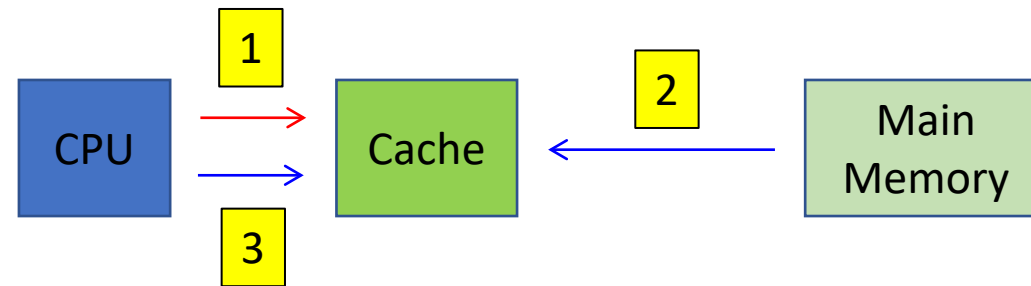
- **Write back** (also called copyback) updates memory **only** when the cache block is selected for replacement.



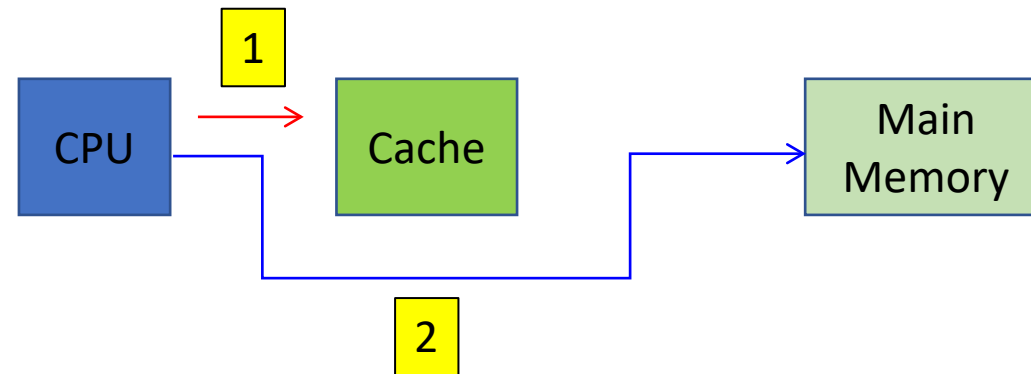
Cache Write Policy for **Write Miss**: CPU wants to write to a certain main memory location but found that it is not in the cache



- **Write allocate** => fetch on write. A write miss will cause the data block at the write-miss address to be **loaded to the cache**.



- **Write-no-allocate** => A write miss will not cause the data block to be loaded to the cache. Data **Write** is done **directly** to its location in **main memory**.





Cache Performance-Related Terminologies

- **Cache Hit**

- Data is found in the cache; either a read hit or a write hit

- **Cache Miss**

- Data is not found in the cache; either a read miss or a write miss

- **Cache Hit rate (H)**

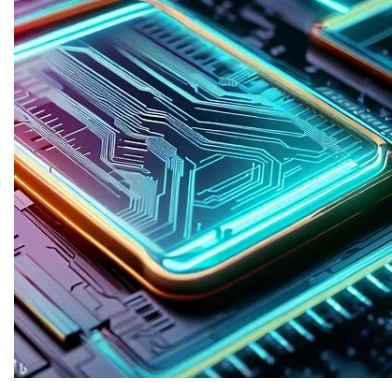
- Percentage of time data found in the cache

- **Cache Miss rate**

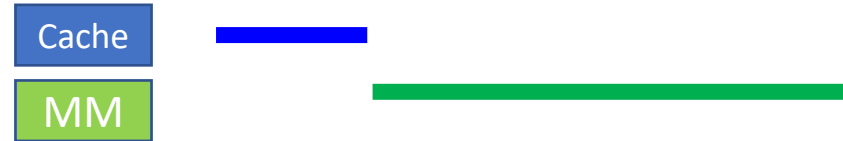
- Percentage of time data not found in cache.

$$\text{Miss rate} = 1 - \text{Hit rate} = 1 - H$$

Effective Access Time



Sequential Access of Cache and Main Memory, i.e. access do not overlap.



- The performance of hierarchical memory is measured by its **effective access time (EAT)**.
- EAT is a **weighted average** that takes into account the hit ratio and relative access times of successive levels of memory.
- The EAT for a two-level memory is given (as a general form):

$$EAT = H \times Access_C + (1-H) \times Miss\ Penalty$$

- $Access_C$ = Access times for cache
- **Miss Penalty** = Time need to access the data when there is **Cache Miss**
- If we assume that data access to Cache and Main memory **do not overlap**, then **Miss Penalty = $Access_C + Access_{MM}$** , where $Access_{MM}$ is the access time for main memory

$$EAT = H \times Access_C + (1-H) \times (Access_C + Access_{MM})$$

applies only to READ operations as write operations will be more complicated (with combination of write-through/back, write-allocate/no-allocate policies)

Effective Access Time (Example)

Sequential Access of Cache
and Main Memory



- Consider a system with a **main memory access time of 200ns** supported by a **cache** having a **10ns access time** and a **hit rate of 99%**.
- If the **accesses do not overlap**,

The EAT is:

$$0.99(10\text{ns}) + 0.01(10\text{ns} + 200\text{ns}) = 9.9\text{ns} + 2.1\text{ns} = 12\text{ns}.$$

- This equation for determining the effective access time can be extended to any number of memory levels.

No part of this material shall be filmed, recorded, downloaded, reproduced, distributed, republished, or transmitted in any form or by any means without written approval from Nanyang Technological University.